

Docker - The Complete Guide

Why Docker Exists (Root Problem)

→ Developer Machine →

- | | |
|---------|-----------------|
| Java 17 | Java 11 |
| Ubuntu | Node 18 |
| Node 18 | Different ports |

→ Server Machine →

- | |
|-----------------|
| Java 11 |
| Amazon Linux |
| Different ports |

❌ Works on my machine → ❌ Fails in production

What Docker Guarantees

- Same App
- Same Dependencies
- Same OS Environment
- Same Config

✓ Same Behavior Everywhere



What Docker Actually Is

NOT

- * Programming Language
- * Framework
- * Virtual Machine

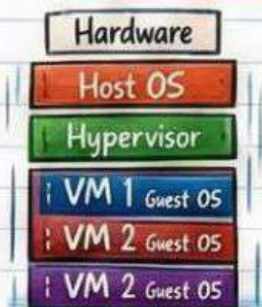


IS

Platform to Package, Ship & Run "Containers"

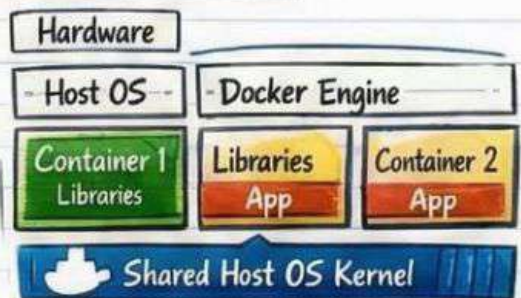


Virtual Machine Model



- * Heavy,
- * Slow Boot,
- * High Memory Usage

Docker Model



What Is a Container (Very Important)

A container is just a process, but with isolation.

- ✓ Its own filesystem
- ✓ Its own process tree.
- ✓ Its own network namespace

Containers ≠ Mini OS

Containers = Isolated Processes



Docker ARCHITECTURE + IMAGE + LAYERS

Docker Architecture (Big Picture)

YOU (Developer)

| docker run, docker build

Docker Client

Docker Daemon (dockerd)

- Image Mgmt
- Container Mgmt
- Network Mgmt
- Volume Mgmt

Linux Kernel



Docker Registry (Where Images Live)

Your Machine

docker pull nginx



Docker Registry
(Docker Hub)



Other Machines / Servers

Registry = Image Store

★ Docker Image (MOST IMPORTANT CONCEPT)

What is an Image?

An image is a read-only template used to create containers.

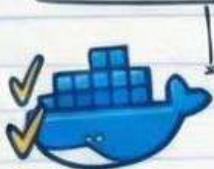
Think:

Image = Class → Container = Object

Image Layering (CRITICAL)

Dockerfile example:

- FROM openjdk:17
- WORKDIR /app
- COPY app.jar ...
- CMD ["java", "-jar", "app.jar"]



- Is cached
- Is reusable
- Is immutable

Layer breakdown:

- ▶ Layer 1: : Base OS
- ▶ Layer 2: : OpenJDK 17
- ▶ Layer 3: : /app directory
- ▶ Layer 4: : app.jar
- ▶ Layer 5: : startup command

- ✓ Is cached
- ✓ Is reusable
- ✓ Is immutable

Build Image & DockerFile

Build Image (Flow)

-  `docker build`
- Image Created
- Stored Locally

`docker build -t myapp.`

Run Container (Flow)

-  `docker run`
- Container Created
- Process Starts

Example: `docker run -p 8080:8080 myapp`

Port Mapping (Very Important)

Host Machine → Container
localhost:8080 → 8080
→ Container 

Syntax: `-p HOST.PORT:CONTAINER.PORT`

Container Lifecycle

Created → Running → Stopped → Removed

-  `docker run`
- create container = start process
-  `docker stop` - process stops
-  `docker rm` - container deleted

Container Commands (Mental Grouping)

- See** `docker ps`
-  `docker ps -a`
- Control** `docker start`
-  `docker stop`
- `docker restart`
- Cleanup** `docker rmi`
- `docker system prune`

Why Volumes Exist (Data Problem)

-  Container
- All data lost
-  Containers are ephemeral

Docker Volumes (Solution)

-  `docker-compose.yml`
- `docker-compose up`
- All containers start together



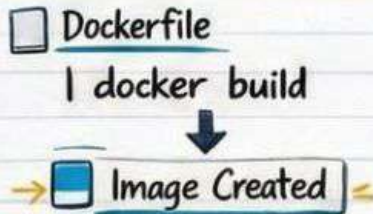
Docker Networking (Beginner Model)



- Containers can talk using container names
- External access requires port mapping

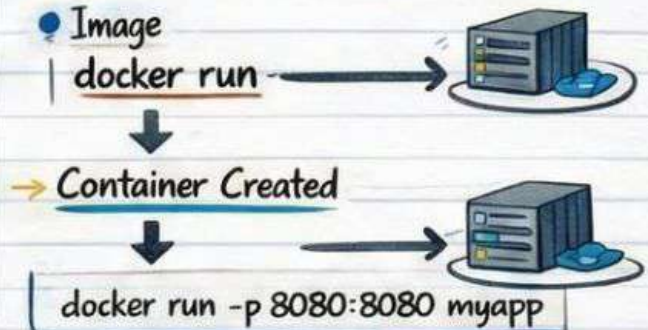
Docker BUILD, RUN, NETWORKING, STORAGE, COMPOSE

Build Image (Flow)



Command: `docker build -t myapp...`

Run Container (Flow)



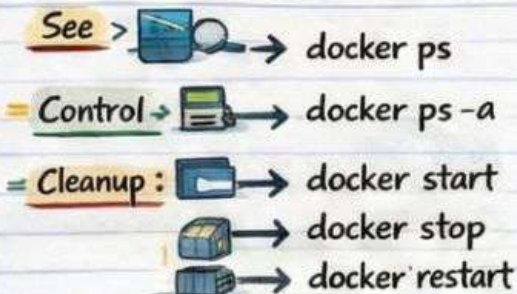
≡ Port Mapping (Very Important) ≡

Host Machine → Container

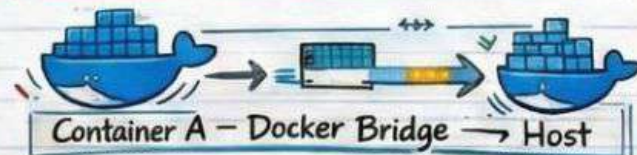
localhost:8080 → 8080

Syntax: `-p HOST_PORT : CONTAINER.PORT`

≡ Container Commands (Mental Grouping) ≡



★ Container Commands (Mental Grouping)



Docker Networking (Beginner Model)

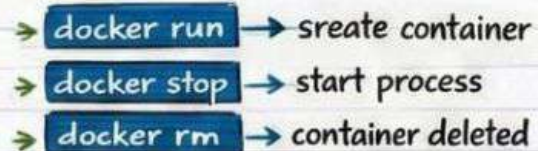
→ Default bridge network

- ★ Containers can talk using comear
- ★ External access requires port mapping

Container Lifecycle

Created → Running → Stopped → Removed

Flow:



★ Docker Volumes (Multi-Container Apps)

★ Problem Container deleted

→ All data lost

Containers are ephemeral

≡ Docker Compose (Multi-Container Apps)

Problem Withoat Compose

- Run DB manually • Run Backend manually
- Run Frontend manually • Mesay.

★ Compose Solution

`docker-compose.yml` 

version →	services →	environment
	• app	• image
	• ports	• app
	• depends.on	• environment

• `docker run -v /host/path : /container/path app`